

Recell startup

Supervised Learning, Foundations

Pauline Zeestraten
January 3, 2024

Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- EDA Results
- Data Preprocessing
- Model Performance Summary
- Appendix

Executive Summary

Based on the final machine learning model, after checking for all assumptions of linear regression, the following independent variables have the most influence over the pricing of normalized used devices:

- `main_camera_mp`
- `selfie_camera_mp`
- `battery`
- `Normalized_new_price`
- `Years_since_release`
- `brand_name_Karbonn`
- `brand_name_LG`
- `brand_name_Micromax`
- `brand_name_Samsung`
- `os_Others`
- `5g_yes`

Recommendation is to hone in on competitive pricing of models of years 2018 and later in particular. Based on our analysis, later used models are not priced as competitively.

Another recommendation is to hone in on specialized markets of customers interested in devices with a certain capacity, i.e. large screen for entertainment, or large battery capacity for use on the road, in order to further optimize sales.

Business Problem Overview and Solution Approach

Business Problem Overview

The used and refurbished device market has grown considerably over the past decade. This growth can be attributed to an uptick in demand for used phones and tablets that offer considerable savings to both consumers and businesses compared with new models. Recell, a startup aiming to tap the potential in this market, has identified the need to develop a dynamic pricing strategy for used and refurbished devices.

Solution approach and methodology

In this report I used a Machine Learning based solution approach to develop such a dynamic pricing strategy. I analyzed the dataset provided by Recell using univariate and bivariate analysis and preprocessed the data where needed. Next I built a linear regression model to predict the price of a used phone/tablet , and identified factors that significantly influence it.

EDA Results - Univariate analysis

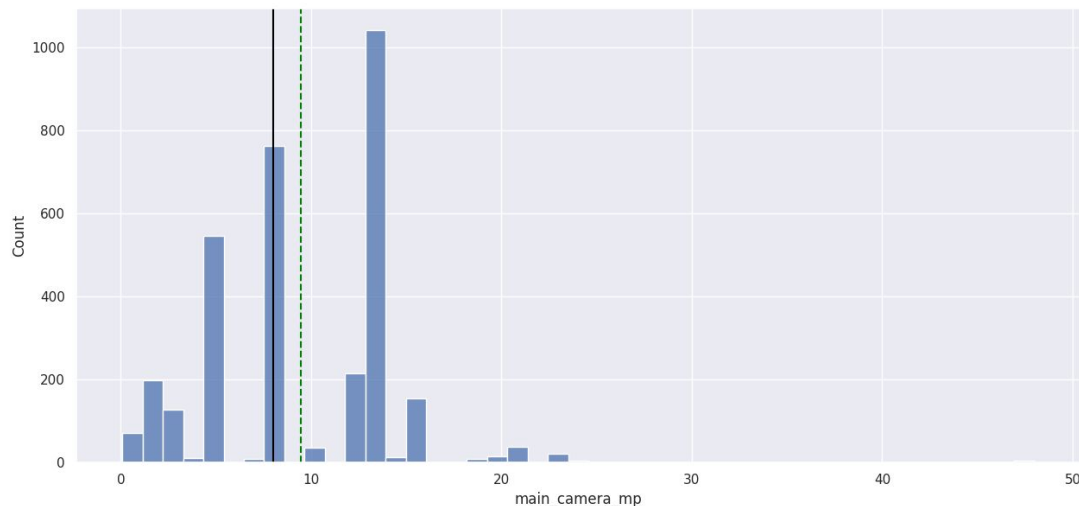
- In the univariate analysis of this dataset, outliers feature heavily in the distribution of data for certain columns or features of the used / refurbished device. See the Appendix for detailed code, graphs and plots. The reasons for outliers can for the majority be explained by the wide variety of features, and the wide range within those features in cell phone and tablet devices in general.

Note: You can use more than one slide if needed

[Link to Appendix slide on data background check](#)

EDA Results - Univariate analysis

Resolution of the rear camera in megapixels - example of outlier effect

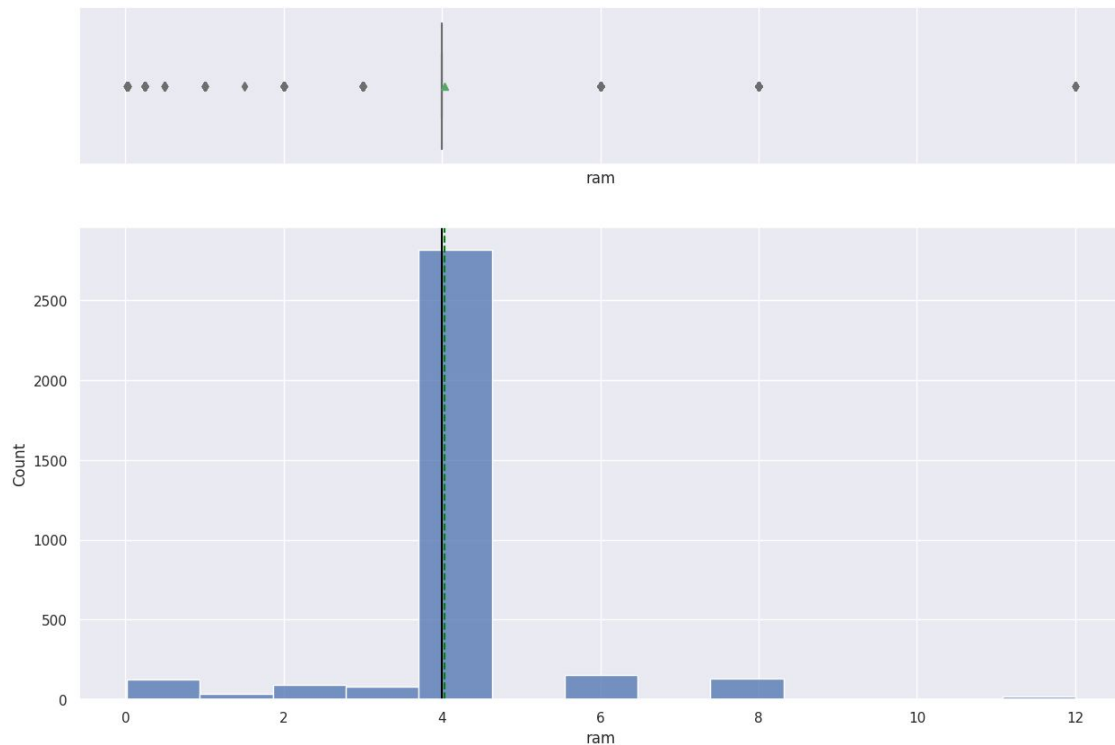


Observation:

Keeping mind that this column has a little over 5% of its data missing, the histo_box plot reveals an oddly distributed graph, skewed to the right with two far outliers. The median is 8 and the mean is 9.46.

EDA Results - Univariate analysis II

Amount of RAM in GB - example of large contrast in concentrated distribution and outliers

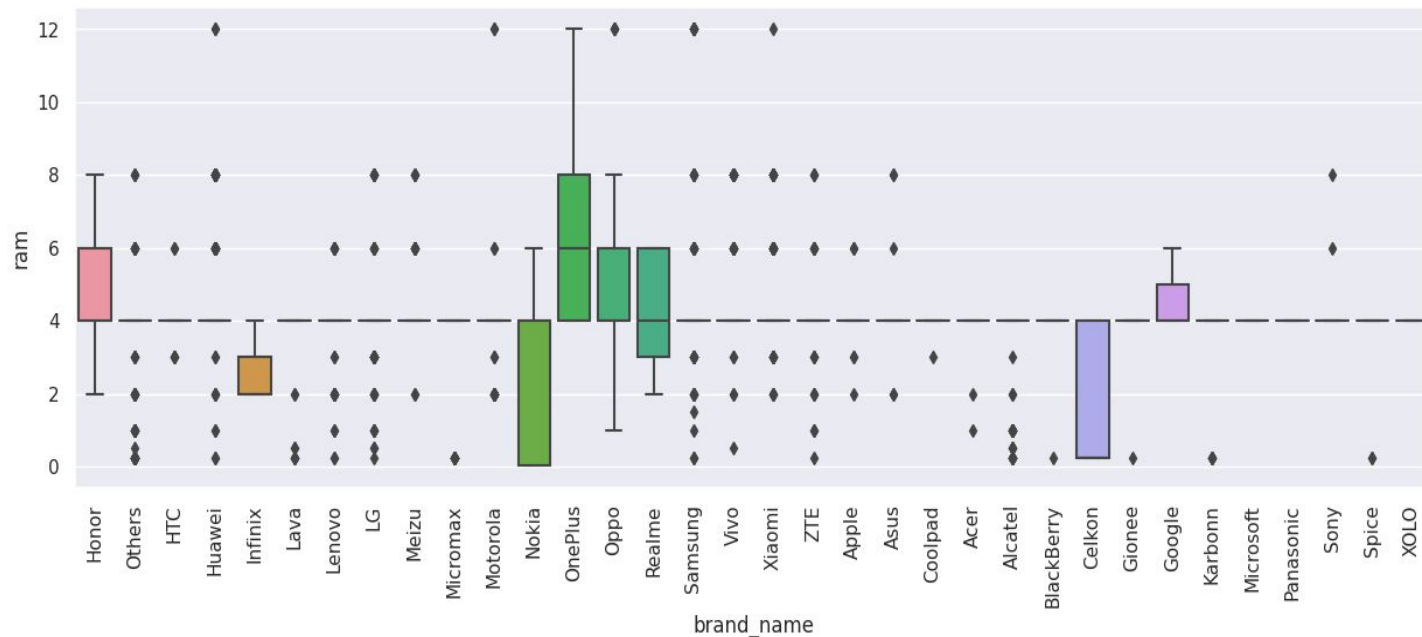


Observation:

This column has 0.11% of its data missing, its impact limited. The histo_box plot reveals the mean, median, and the Interquartile Range(IQR) all centering around 4 RAM. Outliers are found on either tail.

EDA Results - Bivariate analysis

The amount of RAM is important for smooth functioning of a device. Let's see how the amount of RAM varies across brands.



EDA Results - Bivariate analysis

Observations on how the amount of RAM varies across brands:

As we saw earlier in the Univariate Analysis, the amount of RAM provided in used / refurbished devices is 4GB for all the main statistical categories: Mean, median, and Interquartile Range (IQR). However, as we can see here in this boxplot:

The following brands provide significantly **less** than 4GB on their devices:

- 100% of Infinix and Celkon devices.
- 75% of Nokia devices.
- 50% of Realme devices.
- 25% of Honor and Oppo devices.

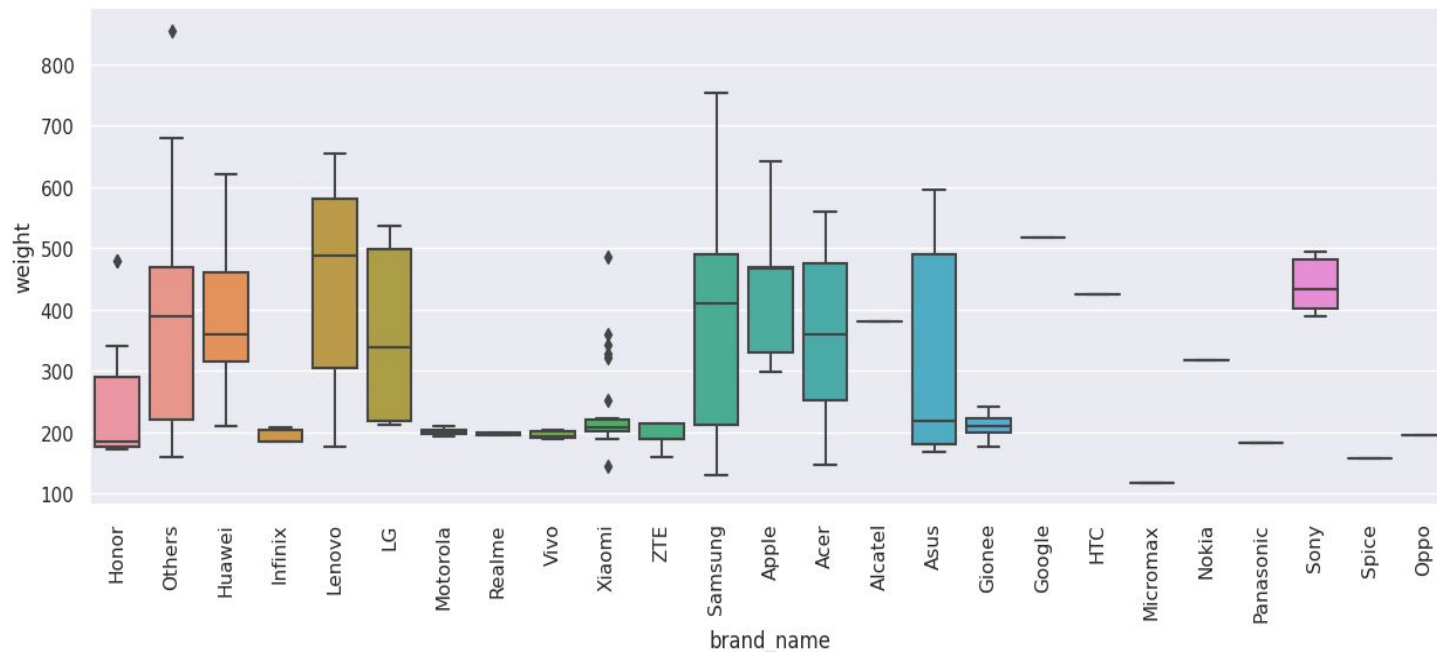
The following brands provide significantly **more** than 4GB on their devices:

- 100% of OnePlus and Google devices
- 75% of Honor and Oppo devices
- 50% of Realme devices
- 25% of Nokia devices

EDA Results - Bivariate analysis II

People who travel require devices with large batteries which often increases weight, making it uncomfortable holding in the hand.

Let's create a new dataframe of devices which offer a large battery, and analyze.



EDA Results - Bivariate analysis II

Observations on analysis of large battery models and weight across brands:

These brands offer devices with large batteries exceeding 4500 mAh, weighing more than 400 grams:

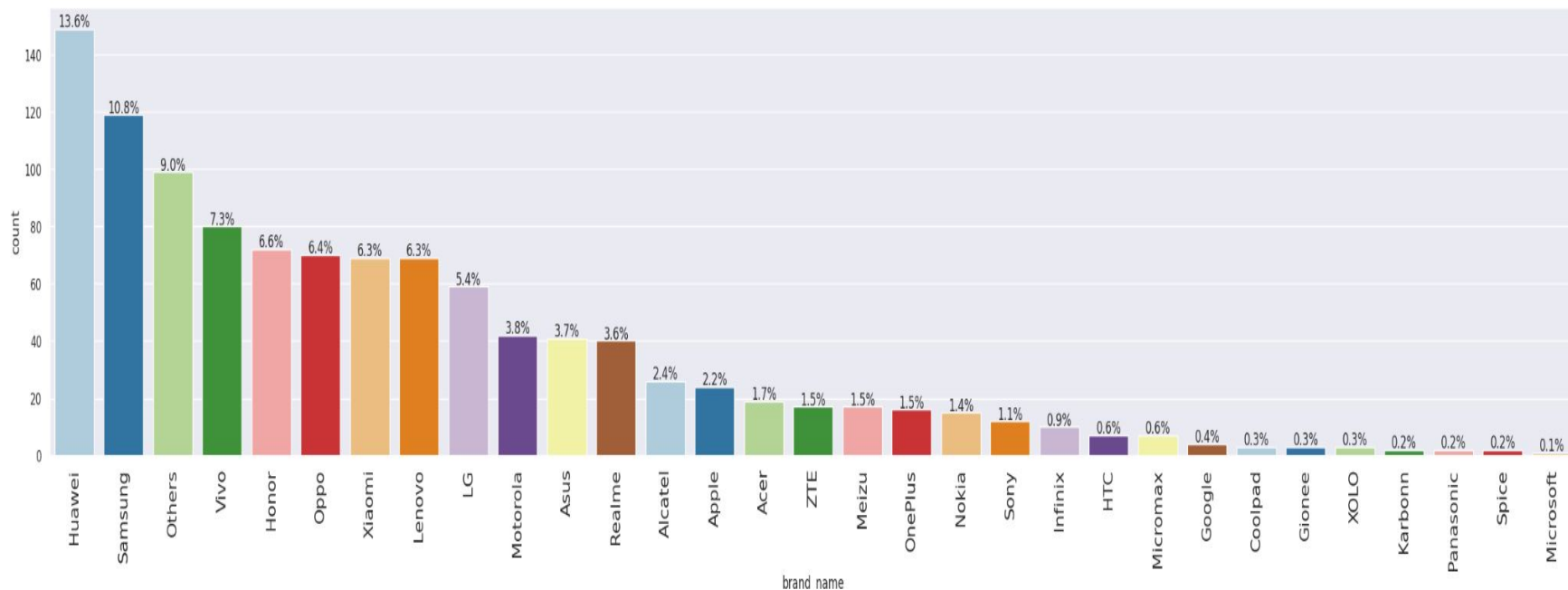
- “Others”, Samsung (50% of devices)
- Lenovo, Apple (>50% of devices)
- Google, HTC, Sony (100% of devices)

These brands offer devices with large batteries exceeding 4500 mAh, weighing less than 400 grams:

- Honor, Infinix, Motorola, Realme, Vivo, ZTE, Gionee, Micromax, Nokia, Panasonic, Spice, Oppo (100%)
- Huawei, LG, Acer, Asus (>50%)

EDA Results - Bivariate analysis III

People who buy phones and tablets for entertainment purposes prefer a large screen.
Let's create a dataframe of only those devices which are suitable for such people and analyze.



EDA Results - Bivariate analysis III

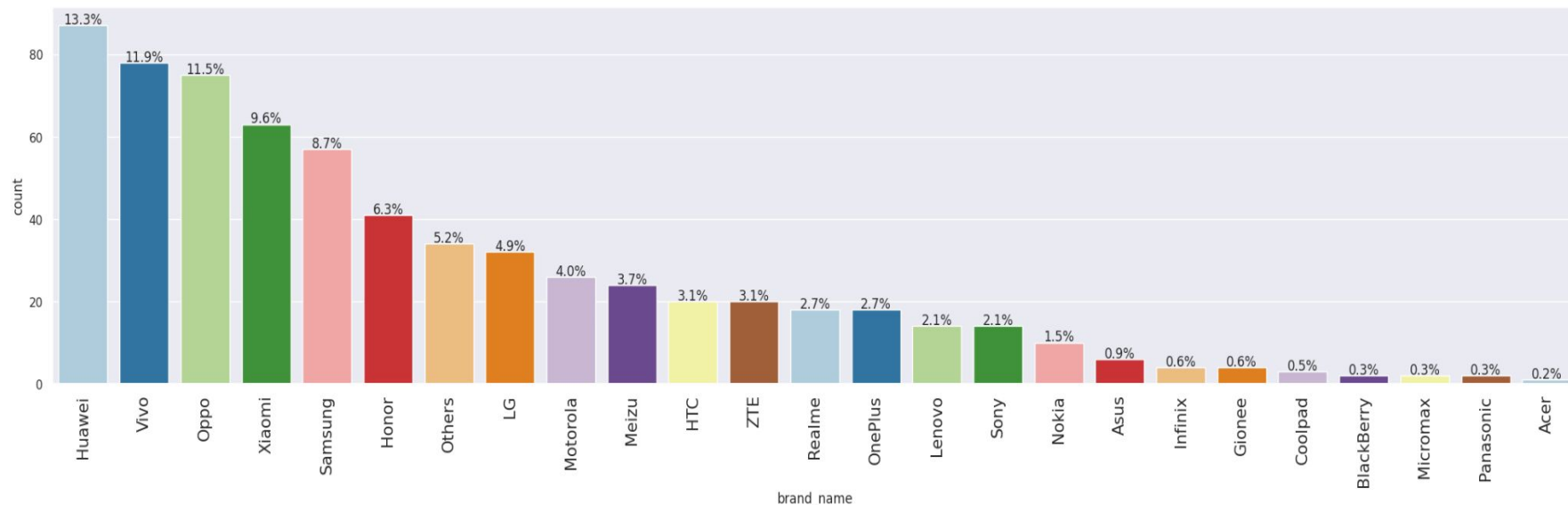
Observations on large screen devices are:

The following brands offer the highest percentage large screen devices:

- Huawei (13.6%)
- Samsung (10.8%)
- Others (9%)
- Vivo (7.3%)

EDA Results - Bivariate analysis IV

Everyone likes a good camera to capture their favorite moments with loved ones. Some customers specifically look for good front cameras to click cool selfies. Let's create a dataframe of those devices which are suitable for this customer segment and analyze.



EDA Results - Bivariate analysis IV

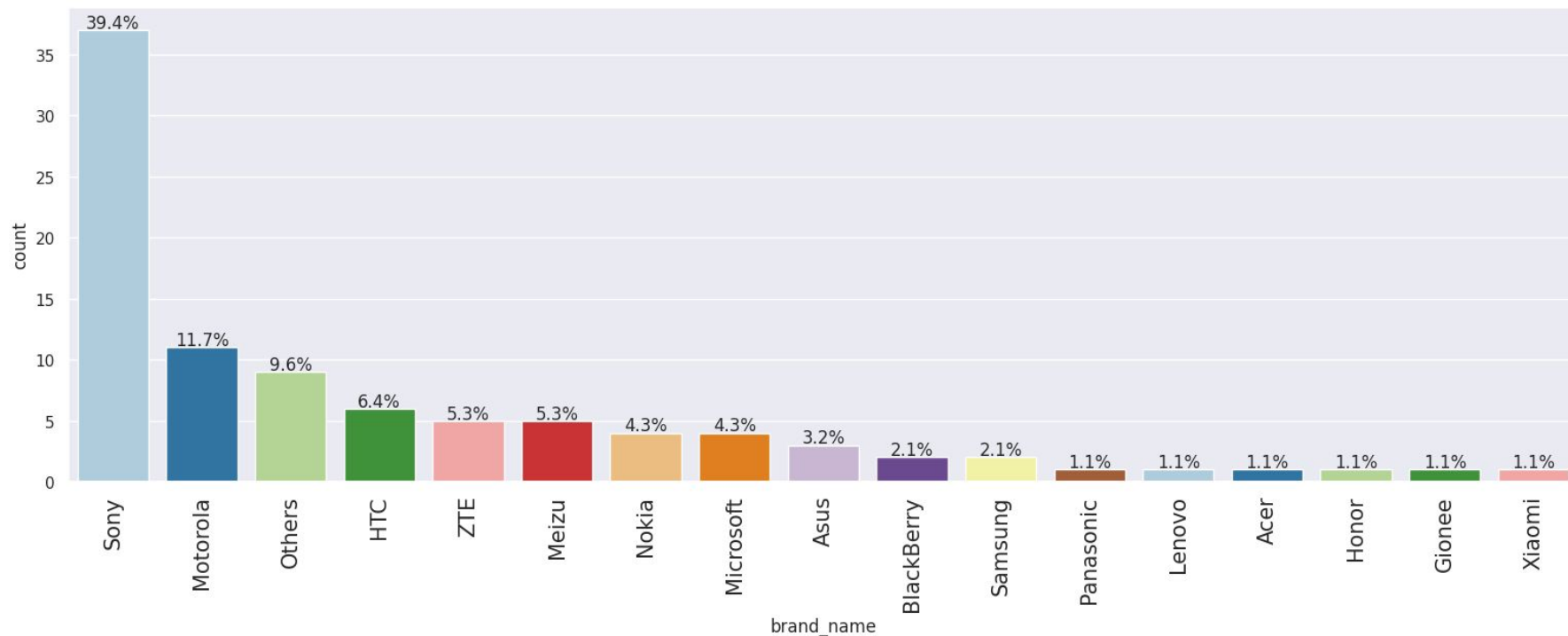
Observation on devices with good front cameras:

The following brands together carry more than 40% off all devices suitable for selfies:

- Huawei (13.3%)
- Vivo (11.9%)
- Oppo (11.5%)
- Xiaomi (9.6%)

EDA Results - Bivariate analysis V

Let's do a similar analysis for rear cameras. Rear cameras generally have a better resolution than front cameras, so we set the threshold higher for them at 16MP.



EDA Results - Bivariate analysis V

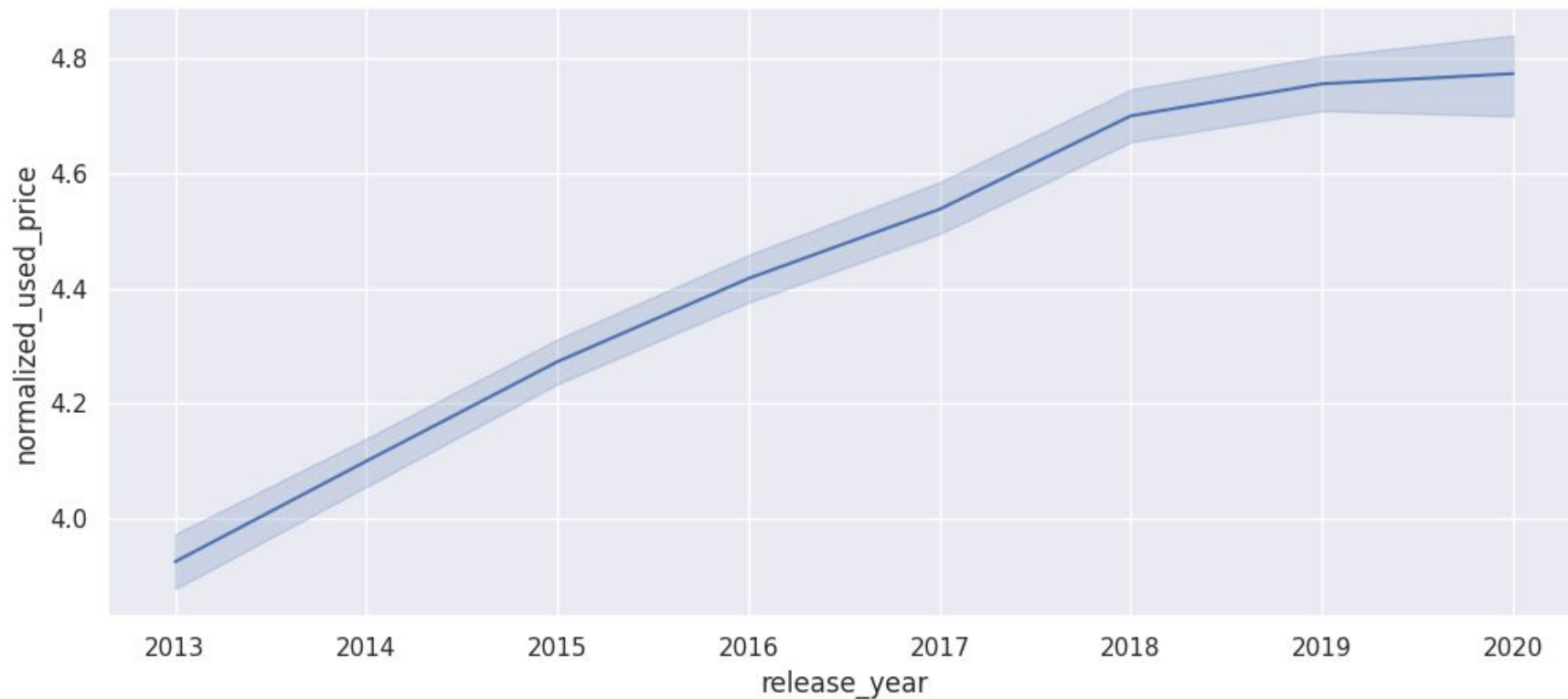
Observation on high resolution rear camera devices:

The following brands together offer more than 40% of all devices with high resolution rear cameras:

- Sony (39.4%)
- Motorola (11.7%)

EDA Results - Bivariate analysis VI

Let's see how the price of used devices varies across the years.



EDA Results - Bivariate analysis VI

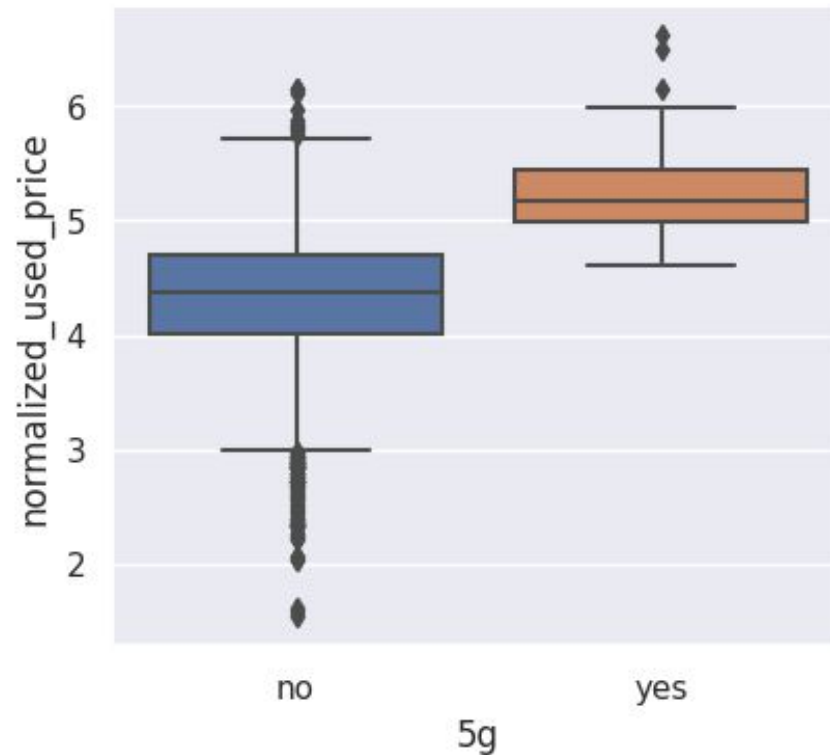
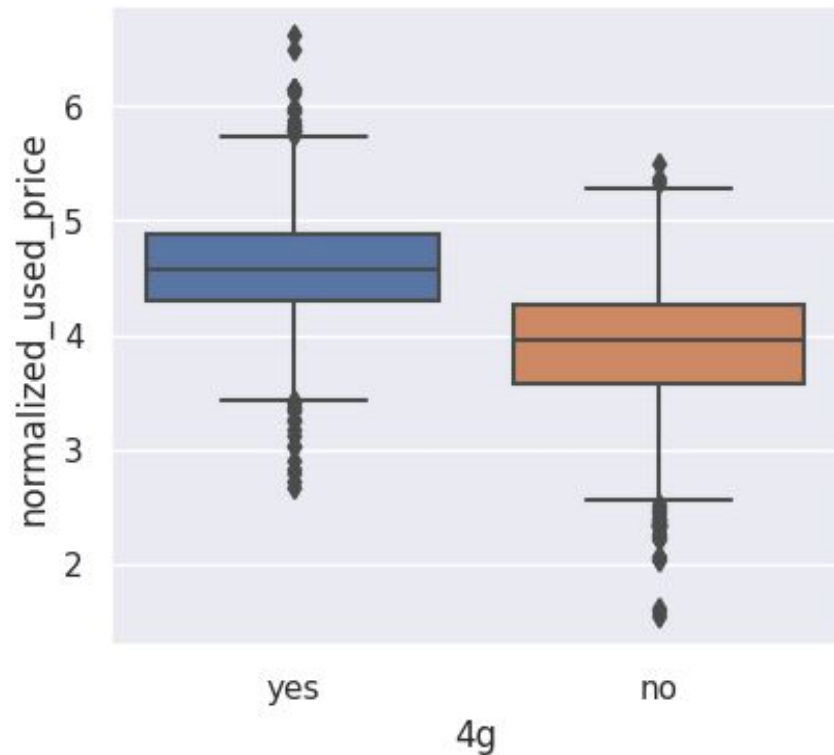
Observation on price of used devices over the years:

The line between price of a used / refurbished device and year of release shows a positive correlation between the years 2013 to 2018.

After 2018 the price increase seems to slightly taper off unexpectedly, when you would predict the demand for more recent used models to be similar if not higher. However this graph shows this is not the case, and this could be an area of focus for future growth.

EDA Results - Bivariate analysis VII

Let's check how prices vary for used phones and tablets offering 4G and 5G networks.



EDA Results - Bivariate analysis VII

Observation on price of used devices offering 4G and 5G networks

- There is no used / refurbished device offering 5G network sold for less than 450 euro at a minimum.
- The price range for used / refurbished devices offering 4G network is between approximately 350 to 575 euro, not taking in consideration outliers on both tails.
- It's hard to draw any conclusions on the no distributions in the two graphs, as it is not further indicated if the devices offer 5G or 4G networks, or if they are of a previous generation all together.

Data Preprocessing - Duplicate and missing values

- Duplicate value check:

```
data.duplicated().sum() ## Complete the code to check duplicate entries in the data
```

There are no duplicate values found in the dataset.

- Missing value treatment

```
data.isnull().mean()*100 ## Complete the code to check duplicate entries in the data
```

Percentage of missing values in the following columns:

main_camera_mp	5.182397
selfie_camera_mp	0.057904
int_memory	0.115808
ram	0.115808
battery	0.173712
weight	0.202664

Data Preprocessing - Missing values treatment

- We will impute the missing values in the data by the column medians grouped by release_year and brand_name.

See appendix for code

- We will impute the remaining missing values in the data by the column medians grouped by brand_name.

```
cols_impute = ["main_camera_mp", "selfie_camera_mp", "battery", "weight",]
```

See appendix for code.

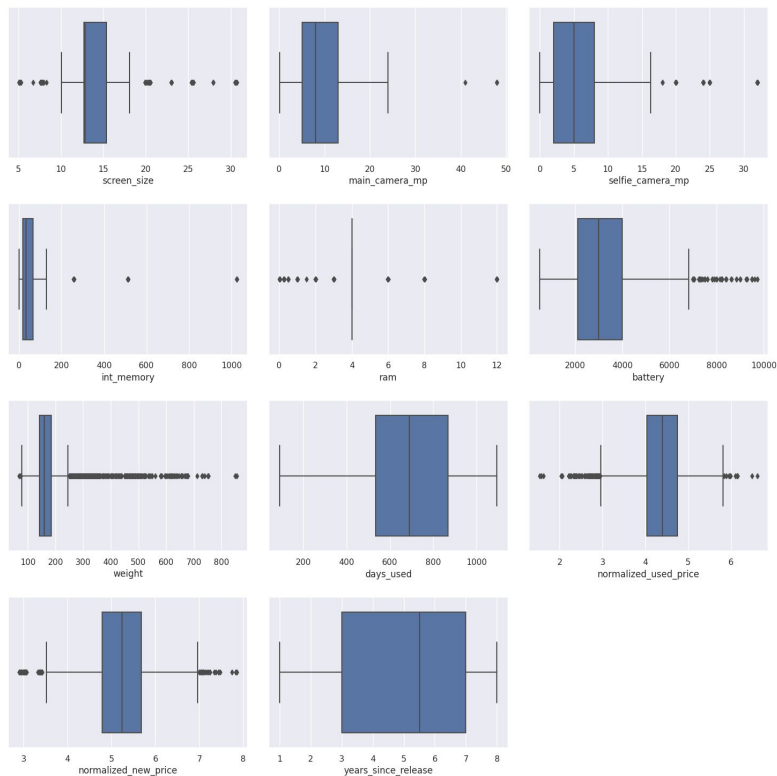
- We will fill the remaining missing values in the main_camera_mp column by the column median.

See appendix for code

Data Preprocessing - Feature engineering

- Feature engineering:
 - Let's create a new column `years_since_release` from the `release_year` column.
 - We will consider the year of data collection, 2021, as the baseline.
 - We will drop the `release_year` column.
 - See appendix for code

Data Preprocessing - Outlier check



Observation:

Outliers are present in nine out of eleven features analyzed.

No treatment is recommended.

The outliers are proper values instead of errors, due to the nature of the wide range and wide variety of features, characteristic of cell phones and tablets.

Data Preprocessing - Data preparation for modeling

- We want to predict the normalized price of used devices
- Before we proceed to build a model, we'll have to encode categorical features
- We'll split the data into train and test to be able to evaluate the model that we build on the train data
- We will build a Linear Regression model using the train data and then check its performance

```
## Complete the code to define the dependent and independent variables
```

```
X = df1.drop(['normalized_used_price'], axis=1)
```

```
y = df1['normalized_used_price']
```

```
print(X.head())
```

```
print()
```

```
print(y.head())
```

```
# let's add the intercept to data
```

```
X = sm.add_constant(X)
```

Data Preprocessing - Data preparation for modeling II

- Before we proceed to build a model, we'll have to encode categorical features

```
# creating dummy variables
```

```
X = pd.get_dummies(  
X,  
columns=X.select_dtypes(include=["object", "category"]).columns.tolist(),  
drop_first=True,  
) ## Complete the code to create dummies for independent features
```

```
X.head()
```

- We'll split the data into train and test to be able to evaluate the model that we build on the train data

```
# splitting the data in 70:30 ratio for train to test data
```

```
x_train, x_test, y_train, y_test = train_test_split(X,y, test_size=0.3, random_state=1) ## Complete the  
code to split the data into train and test in specified ratio
```

Building a linear regression / ML model - overview

- We will build a Linear Regression model using the train data and then check it's performance

```
olsmodell = sm.OLS(y_train, x_train).fit() ## Complete the code to fit OLS model  
print(olsmodell.summary())
```

Interpreting the Regression Results:

Adjusted R-squared: It reflects the fit of the model. Adjusted R-squared values generally range from 0 to 1, where a higher value generally indicates a better fit, assuming certain conditions are met. In our case, the value for adj. R-squared is 0.842, which is good.

const coefficient: It is the Y-intercept. It means that if all the predictor variable coefficients are zero, then the expected output (i.e., Y) would be equal to the const coefficient. In our case, the value for const coefficient is 1.3156

Coefficient of a predictor variable: It represents the change in the output Y due to a change in the predictor variable (everything else held constant). In our case, the coefficient of screen_size is 0.0244.

ML Model Performance Summary

```
# checking model performance on train set (seen 70% data)
print("Training Performance\n")
olsmodell_train_perf = model_performance_regression(olsmodell1, x_train, y_train)
olsmodell_train_perf
```

Training Performance

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	0.229884	0.180326	0.844886	0.841675	4.326841

[Link to Appendix slide on model assumptions](#)

ML Model Performance Summary II

```
# checking model performance on test set (seen 30% data)
print("Test Performance\n")

olsmodell_test_perf = model_performance_regression(olsmodell, x_test, y_test) ## Complete the code to
check the performance on test data
olsmodell_test_perf
```

Test Performance

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	0.238358	0.184749	0.842479	0.834659	4.501651

ML Model Performance Summary III

Observations on the first Linear Regression model without checking assumptions first.:

- The training R^2 is 0.844, so the model is not underfitting
- The train and test RMSE and MAE are comparable, so the model is not overfitting either.
- MAE suggests that the model can predict the `normalized_used_price` within a mean error of 0.184 on the test data.
- MAPE of 4.50 on the test data means that we are able to predict within 4.5% of the `normalized_used_price`.

ML Model Assumptions and test for first assumption

We will be checking the following Linear Regression assumptions:

- No Multicollinearity
- Linearity of variables
- Independence of error terms
- Normality of error terms
- No Heteroscedasticity

We will test for first Linear Regression assumption: multicollinearity using the VIF test.

```
#Let's define a function to check VIF.  
def checking_vif(predictors):  
    vif = pd.DataFrame()  
    vif["feature"] = predictors.columns  
    # calculating VIF for each feature  
    vif["VIF"] = [variance_inflation_factor(predictors.values, i)  
                  for i in range(len(predictors.columns))]  
    return vif
```


ML Model Assumptions - Test for Multicollinearity

- Applying VIF test to check for Multicollinearity:

```
checking_vif(x_train) ## Complete the code to check VIF on train data
```

Observations:

- Columns with a VIF score greater than 5 are: os_iOS, screen size, and weight.
- Columns with a VIF score greater than 5 to be ignored: the constant and the following dummy variables: brand_name_Apple, brand_name_Huawei, brand_name_Samsung, brand_name_Others

Model Assumptions - Multicollinearity removal

1. Removing Multicollinearity by dropping column one by one with a VIF score greater than 5.:
2. Look at the adjusted R-squared and RMSE of all these models.
3. Drop the variable that makes the least change in adjusted R-squared.
4. Check the VIF scores again.
5. Continue till you get all VIF scores under 5.

```
col_list = ['screen_size', 'weight', 'os_iOS'] ## Complete the code to specify the columns with high VIF
```

```
res = treating_multicollinearity(x_train, y_train, col_list) ## Complete the code to check the effect on  
model performance after dropping specified columns from train data
```

Res

col	Adj. R-squared after_dropping col		RMSE after dropping col
0	os_iOS	0.841795	0.232211
1	screen_size	0.838381	0.234703
2	weight	0.838071	0.234928

- See appendix for continuation of code and results of eliminating multicollinearity through VIF testing.

2nd Model Performance (after eliminating multicollinearity, 1st assumption of linear regression)

Observations on the second linear regression model after eliminating multicollinearity and dropping predictor variables with no significant impact on the target variable.

- In this new model no feature has a p-value greater than 0.05, so we'll consider the features in `x_train5` as the final set of predictor variables and `olsmod2` as the final model to move forward with.
- The adjusted R-squared is now 0.820, i.e., our model is able to explain ~82% of the variance. The adjusted R-squared in `olsmod1` (where we considered the variables without multicollinearity) was 0.834. This shows that the variables we dropped were not affecting the model.
- RMSE and MAE values are comparable for train and test sets, indicating that the model is not overfitting.

Checking for linear regression assumption #2: Test for linearity and independence

- We will test for linearity and independence by making a plot of fitted values vs residuals and checking for patterns.

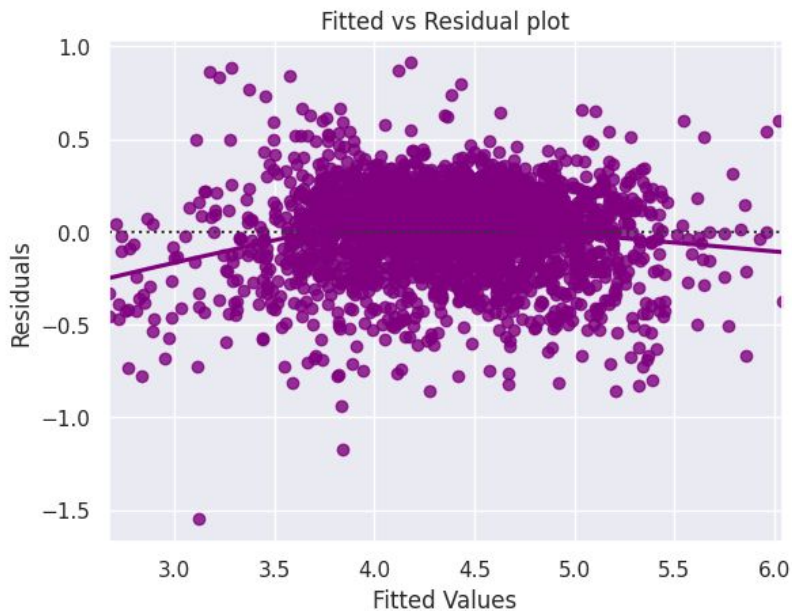
```
# let us create a dataframe with actual, fitted and residual values
df_pred = pd.DataFrame()

df_pred["Actual Values"] = y_train # actual values
df_pred["Fitted Values"] = olsmod2.fittedvalues # predicted values
df_pred["Residuals"] = olsmod2.resid # residuals
# let's plot the fitted values vs residuals

sns.residplot(
    data=df_pred, x="Fitted Values", y="Residuals", color="purple", lowess=True
)
plt.xlabel("Fitted Values")
plt.ylabel("Residuals")
plt.title("Fitted vs Residual plot")
plt.show()
```

Checking for linear regression assumption #2: Test for linearity and independence

- If there is no pattern, then we say the model is linear and residuals are independent.
- Otherwise, the model is showing signs of non-linearity and residuals are not independent.



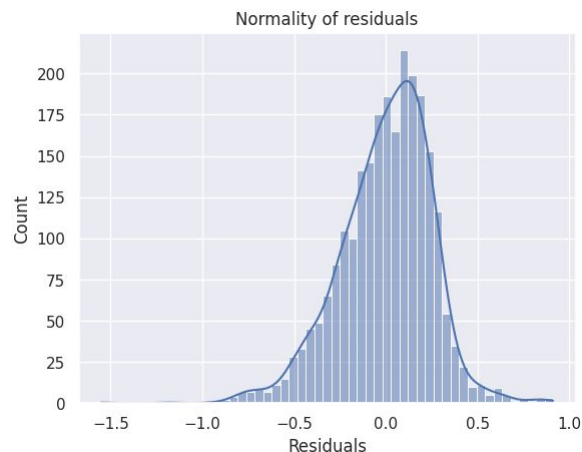
Observation:

- We see no pattern in the residual plot. Hence, the assumptions of linearity and independence are satisfied.

Checking for linear regression assumption #3: Test for normality

- We will test for normality by checking the distribution of residuals, by checking the Q-Q plot of residuals, and by using the Shapiro-Wilk test.

```
sns.histplot(data=df_pred, x="Residuals", kde=True) ## Complete the code to plot the distribution of residuals  
plt.title("Normality of residuals")  
plt.show()
```

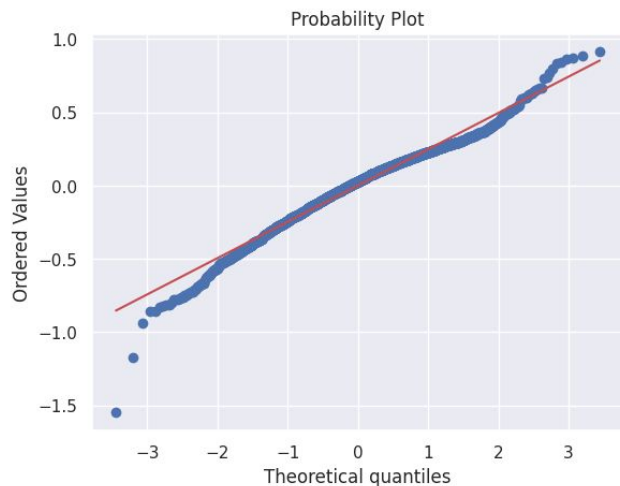


Observation:

- The residuals look fairly normally distributed based on this histogram. We will check the Q-Q plot and use the Shapiro-Wilk test next.

Checking for linear regression assumption #3: Test for normality - Q-Q plot and Shapiro-Wilk test

```
import pylab
import scipy.stats as stats
stats.probplot(df_pred['Residuals'], dist="norm", plot=pylab) ## Complete the code check Q-Q plot
plt.show()
```



Observation Q-Q plot:

- The residuals more or less follow a straight line except for the tails. The assumption of normality is reasonably satisfied

Shapiro-Wilks test:

```
stats.shapiro(df_pred['Residuals'])
ShapiroResult(statistic=0.9784474968910217,
pvalue=1.1490139448281421e-18)
```

- The p-value is < 0.05 so the residuals are not normally distributed according to the S-W test. However, as an approximation, we can accept this distribution as close to being normal.

Checking for linear regression assumption #4: Test for Homoscedasticity

- We will test for homoscedasticity by using the Goldfeldquandt test.
- If we get a p-value greater than 0.05, we can say that the residuals are homoscedastic. Otherwise, they are heteroscedastic.

```
import statsmodels.stats.api as sms
from statsmodels.compat import lzip
```

```
name = ["F statistic", "p-value"]
```

```
test = sms.het_goldfeldquandt(df_pred["Residuals"], x_train5) ## Complete the code with the right train
data to apply the Goldfeldquandt test
```

```
lzip(name, test)
```

```
[('F statistic', 1.0013059518003375), ('p-value', 0.4909982668502599)]
```

- Observation: Since $p\text{-value} > 0.05$, we can say that the residuals are homoscedastic. So, this assumption is satisfied.

Final ML Model summary, after all assumptions tested

```
x_train_final = x_train5
x_test_final = x_test5
olsmodel_final = sm.OLS(y_train, x_train_final).fit() ## Complete the code to fit the final model
print(olsmodel_final.summary())

# checking model performance on train set (seen 70% data)
print("Training Performance\n")

olsmodel_final_train_perf = model_performance_regression(olsmodel_final, x_train_final, y_train) ##
Complete the code to check the performance on train data
olsmodel_final_train_perf
```

Training Performance

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	0.250287	0.194316	0.81613	0.815135	4.677589

Final ML Model summary, after all assumptions tested

```
# checking model performance on test set (seen 30% data)
print("Test Performance\n")

olsmodel_final_test_perf = model_performance_regression(olsmodel_final, x_test_final, y_test) ## Complete
the code to check performance on test data

Olsmodel_final_test_perf
```

Test Performance

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	0.252521	0.195275	0.823203	0.820956	4.778809

Final ML Model observations

Observations on the final linear regression model after eliminating multicollinearity and dropping predictor variables with no significant impact on the target variable.

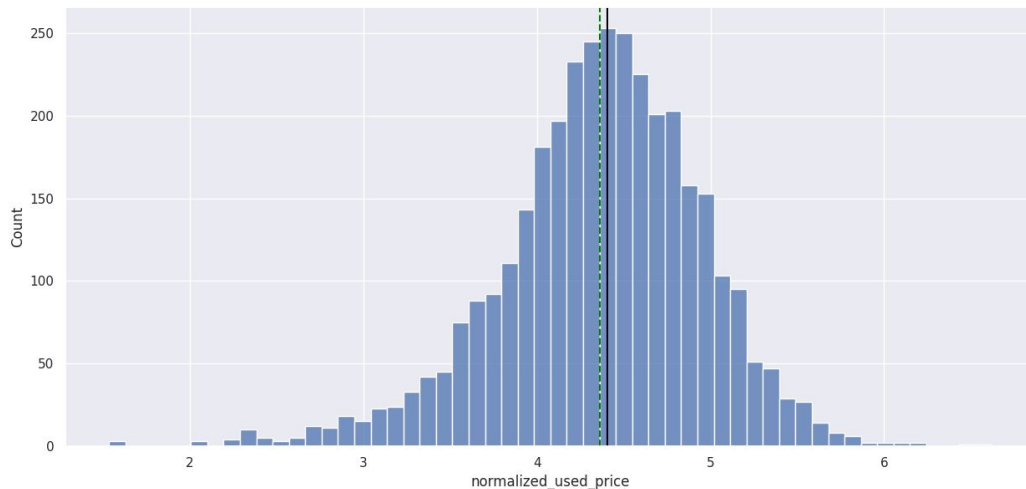
- In this new model no feature has a p-value greater than 0.05, so we'll consider the features in `x_train5` as the final set of predictor variables and `olsmod2` as the final model to move forward with.
- The adjusted R-squared is now 0.820, i.e., our model is able to explain ~82% of the variance. The adjusted R-squared in `olsmod1` (where we considered the variables without multicollinearity) was 0.834. This shows that the variables we dropped were not affecting the model.
- RMSE and MAE values are comparable for train and test sets, indicating that the model is not overfitting.

APPENDIX

Detailed Exploratory Data Analysis - Univariate Analysis

Normalized price of the used / refurbished device in euros

```
histogram_boxplot(df, "normalized_used_price")
```

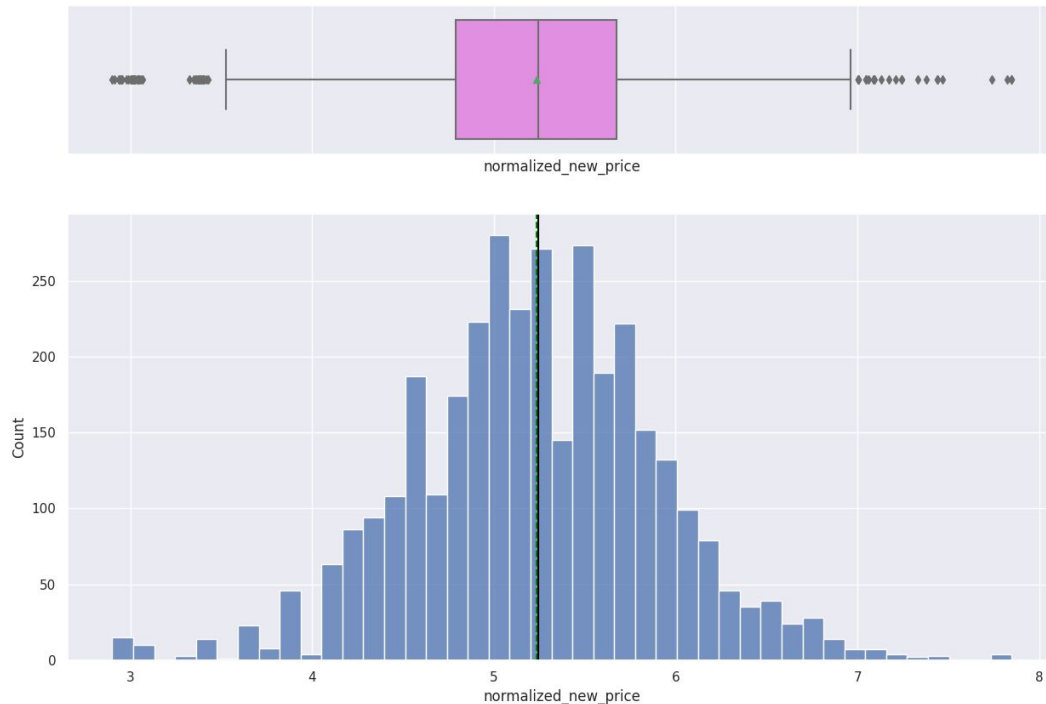


Observation:

The normalized price of used / refurbished devices seems symmetrically distributed, with the exception of a left-skewed tail. The mean and median are very similar and centered at 436 and 440 respectively.

Normalized price of a new device of the same model in Euros

```
histogram_boxplot(df, 'normalized_new_price')
```

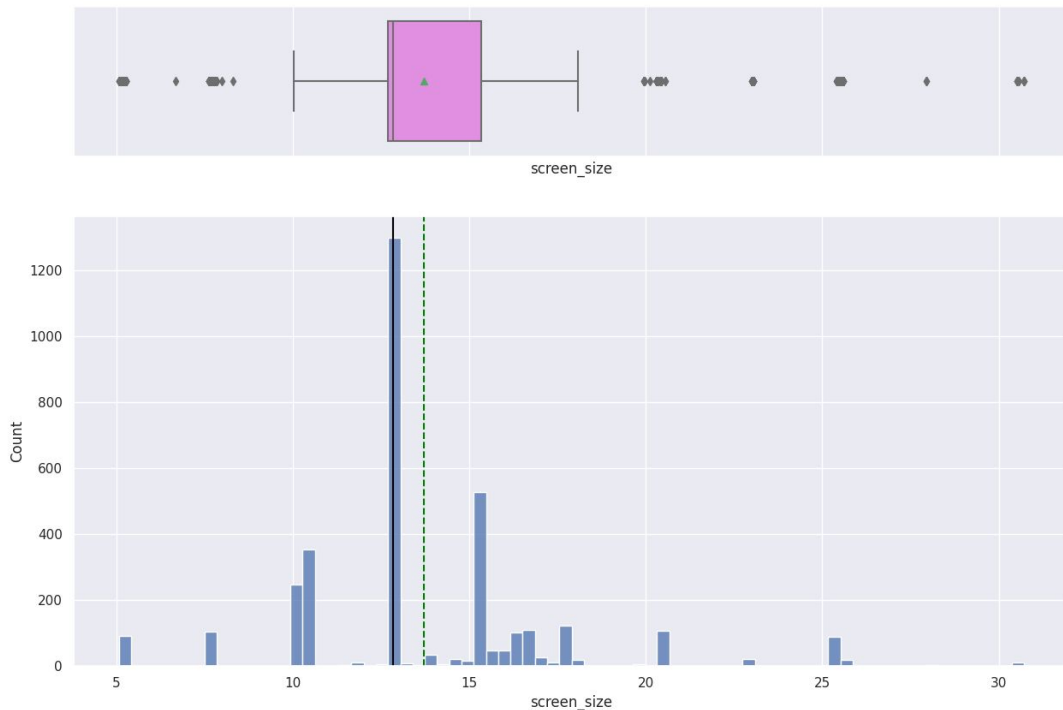


Observation:

The histogram reveals a fairly normally / symmetrically distributed graph, with a few dips and peaks. The boxplot informs us that the median and mean are very close at 524 and 523 respectively, and both tails have outliers.

Size of the screen in centimeters

```
histogram_boxplot(df, 'screen_size')
```

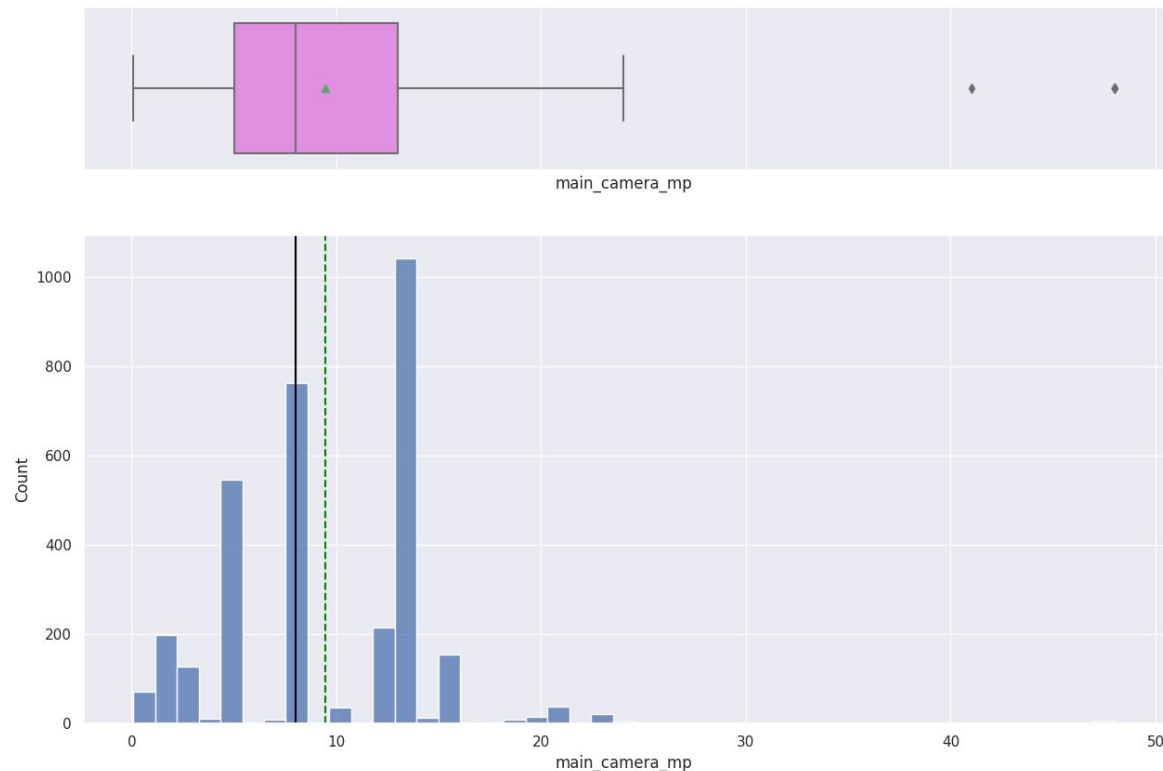


Observation:

The histogram shows an uneven distribution with one peak. The boxplot shows the data distribution is heavily skewed to the right, probably because tablet devices with larger screen sizes are included in the data. The median is 12.8 cm, and the mean 13.7 cm. Outliers on both sides of the distribution.

Resolution of the rear camera in megapixels

```
histogram_boxplot(df, 'main_camera_mp')
```

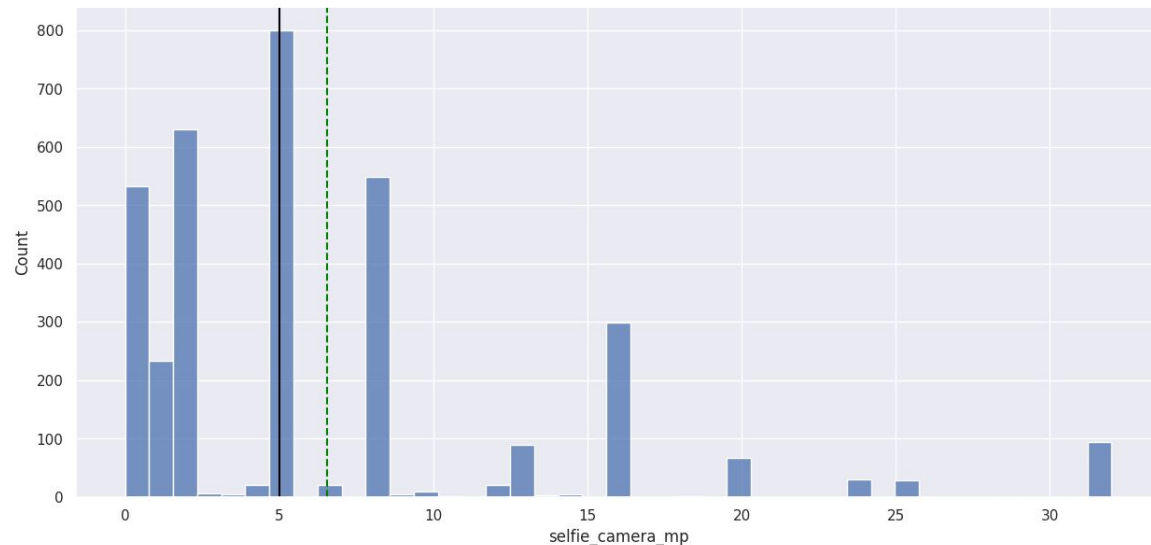
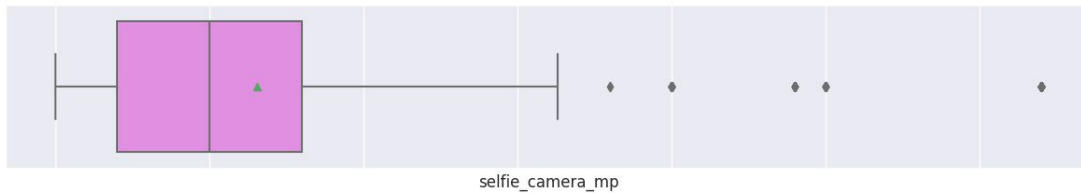


Observation:

Keeping mind that this column has about 5% of its data missing, the histo_box plot reveals an oddly distributed graph, skewed to the right with two far outliers. The median is 8 and the mean is 9.46.

Resolution of the front camera in megapixels

```
histogram_boxplot(df, 'selfie_camera_mp')
```

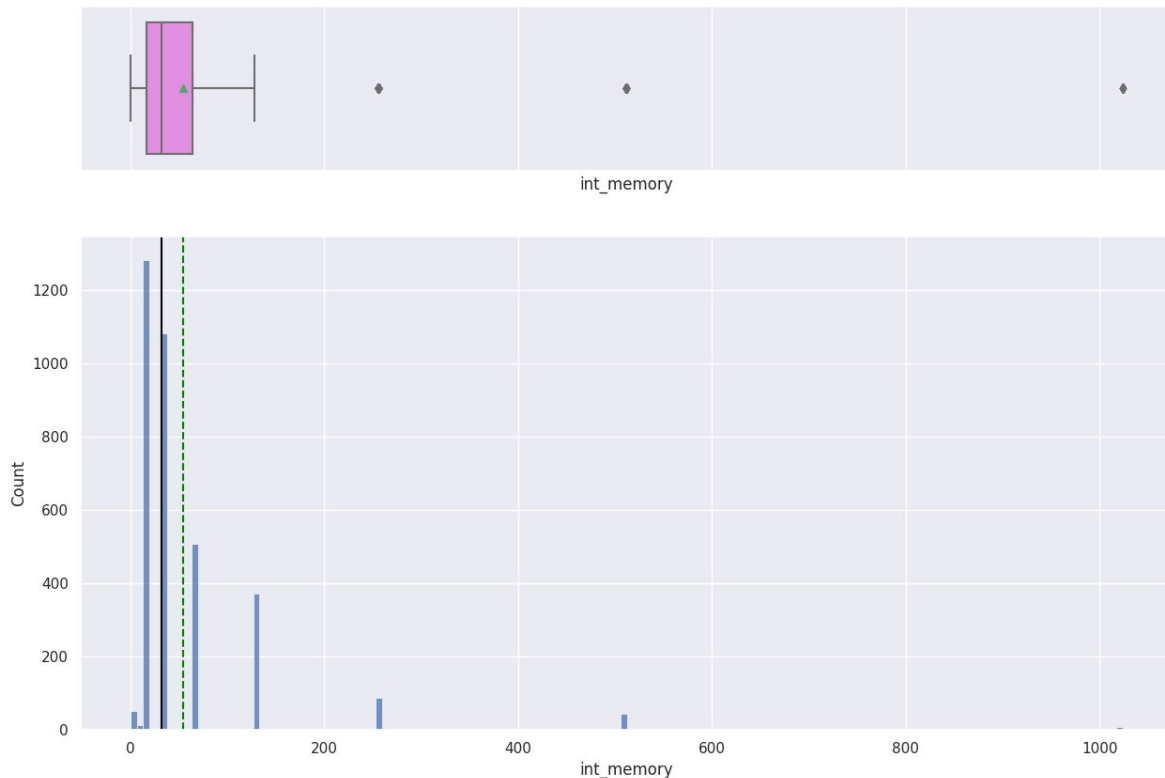


Observation:

Data is missing in this column as well, and the histo_boxplot reveals a right skewed distribution with few outliers. The median is 5 megapixels and the mean is 6.55

Amount of internal memory (ROM) in GB

```
histogram_boxplot(df, 'int_memory')
```

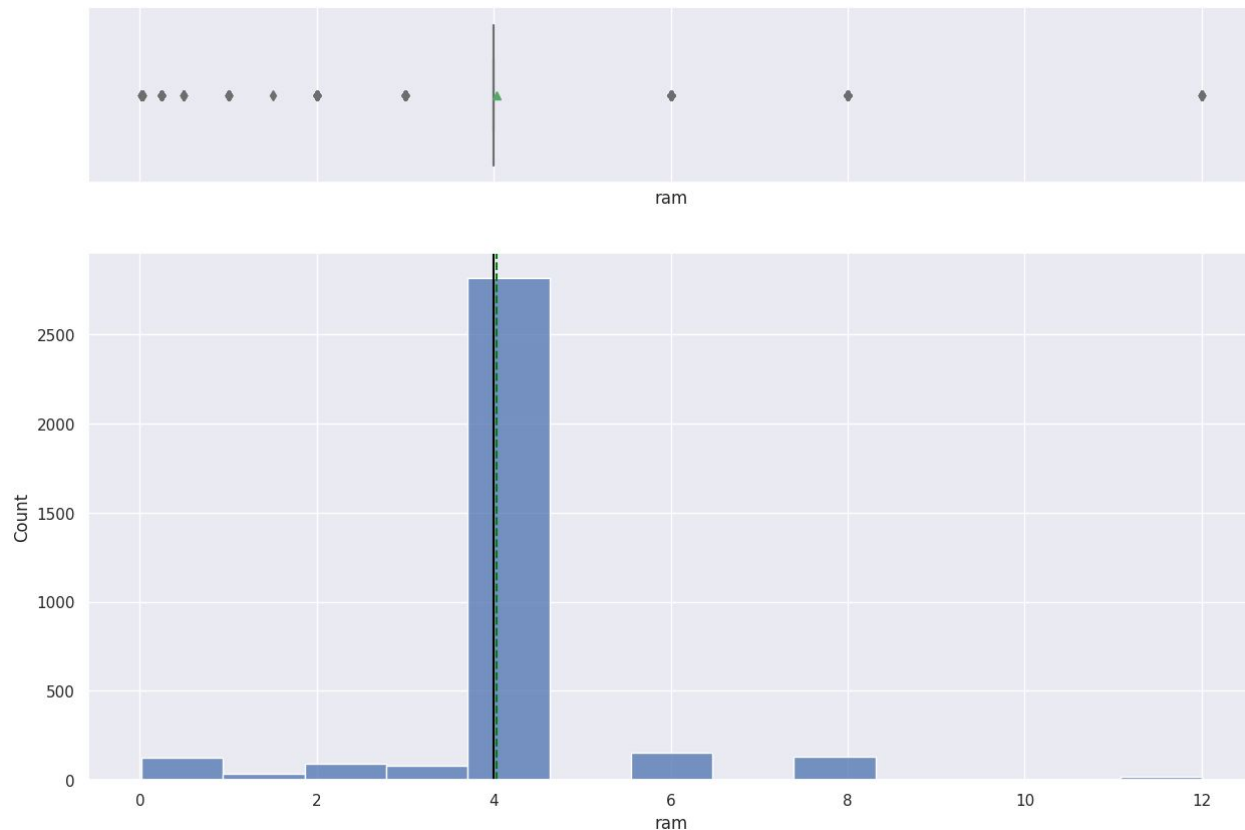


Observation:

The distribution skews far to the right due to outliers. Keep in mind that here too, values are missing. IQR (25 - 75% range) is between 16 to 64. The median is 32 GB, the mean is 54.5 GB. The maximum value is 1024GB.

Amount of RAM in GB

```
histogram_boxplot(df, 'ram')
```



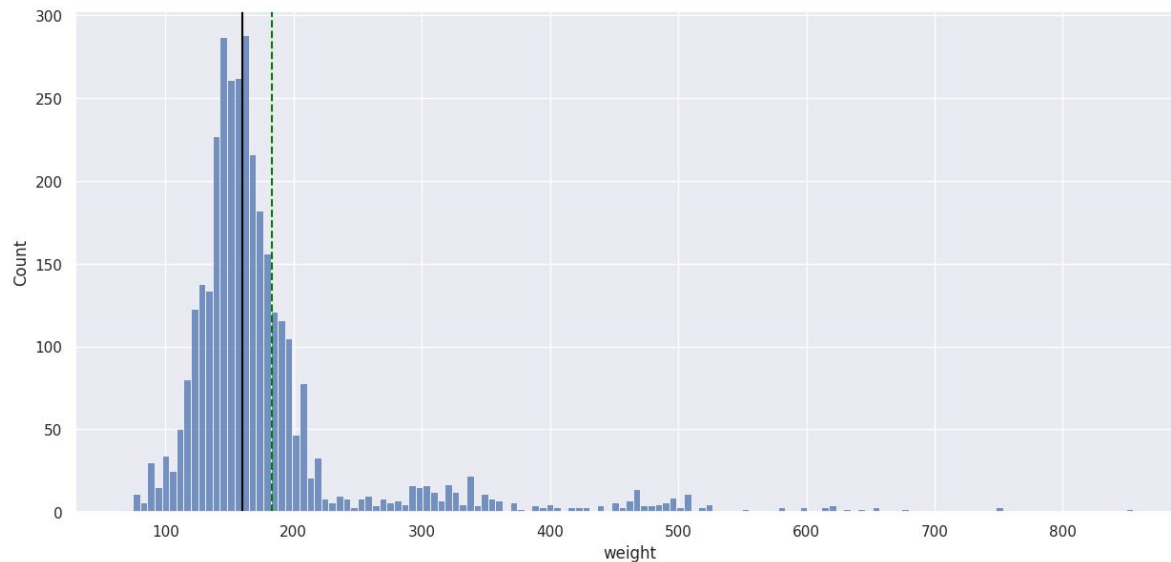
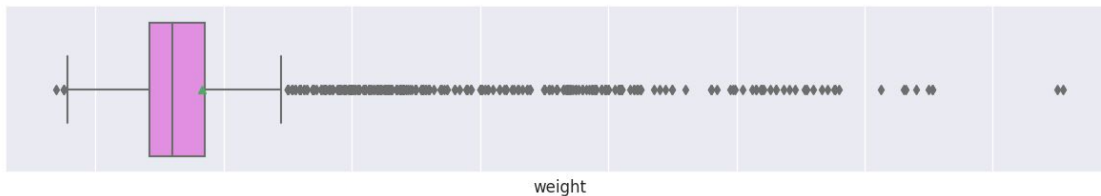
Proprietary content. © Great Learning. All Rights Reserved. Unauthorized use or distribution prohibited.

Observation:

This column has data missing, and the histo_box plot reveals the mean, median, and the Interquartile Range(IQR) all centering around 4 RAM. Outliers are found on either tail.

Weight of the device in grams

```
histogram_boxplot(df, 'weight')
```

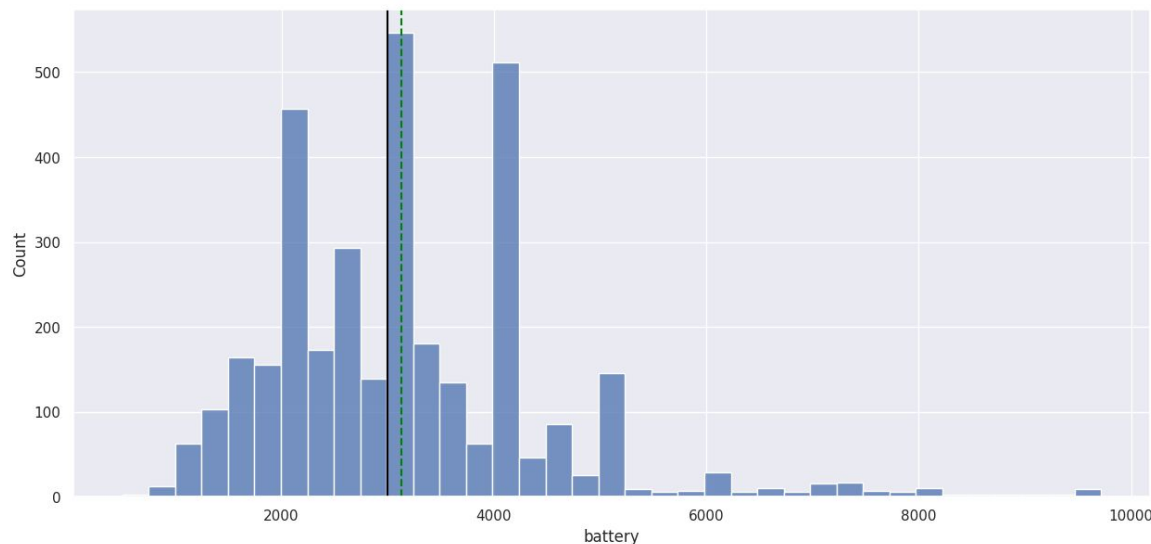
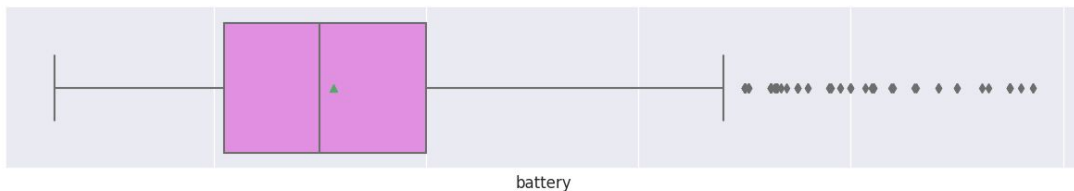


Observation:

The column for weight is missing data. The histo_boxplot reveals a fairly symmetric distribution with a heavily right skewed tail with outliers ranging to the maximum weight of 855 grams. IQR is in the 142 to 185 range with a median of 160 and a mean of 182.75.

Energy capacity of the device battery in mAh

```
histogram_boxplot(df, 'battery')
```

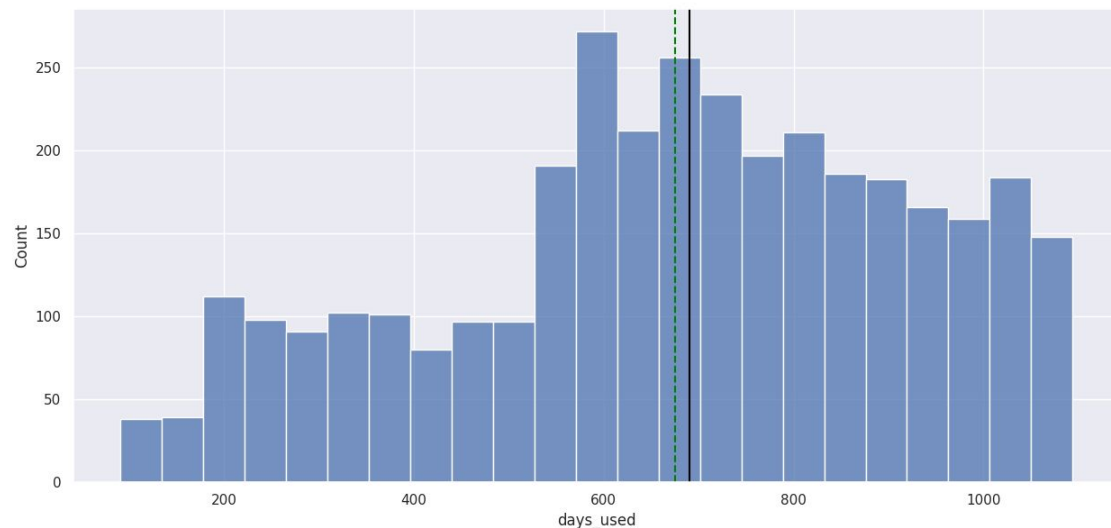
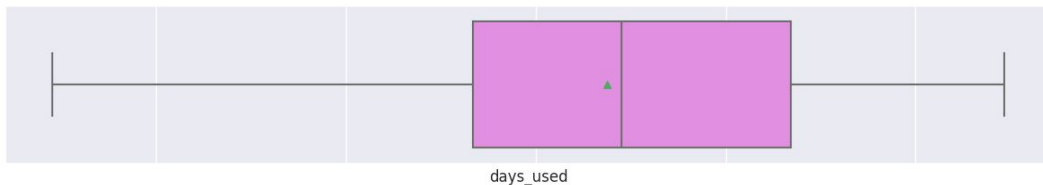


Observation:

Data is missing in this column, but the histo_box plot shows a right skewed distribution of battery capacity. The median is 3000, the mean is 3133. The minimum capacity is 500, while maximum is 9720

Number of days the used / refurbished device has been used

```
histogram_boxplot(df, 'days_used')
```

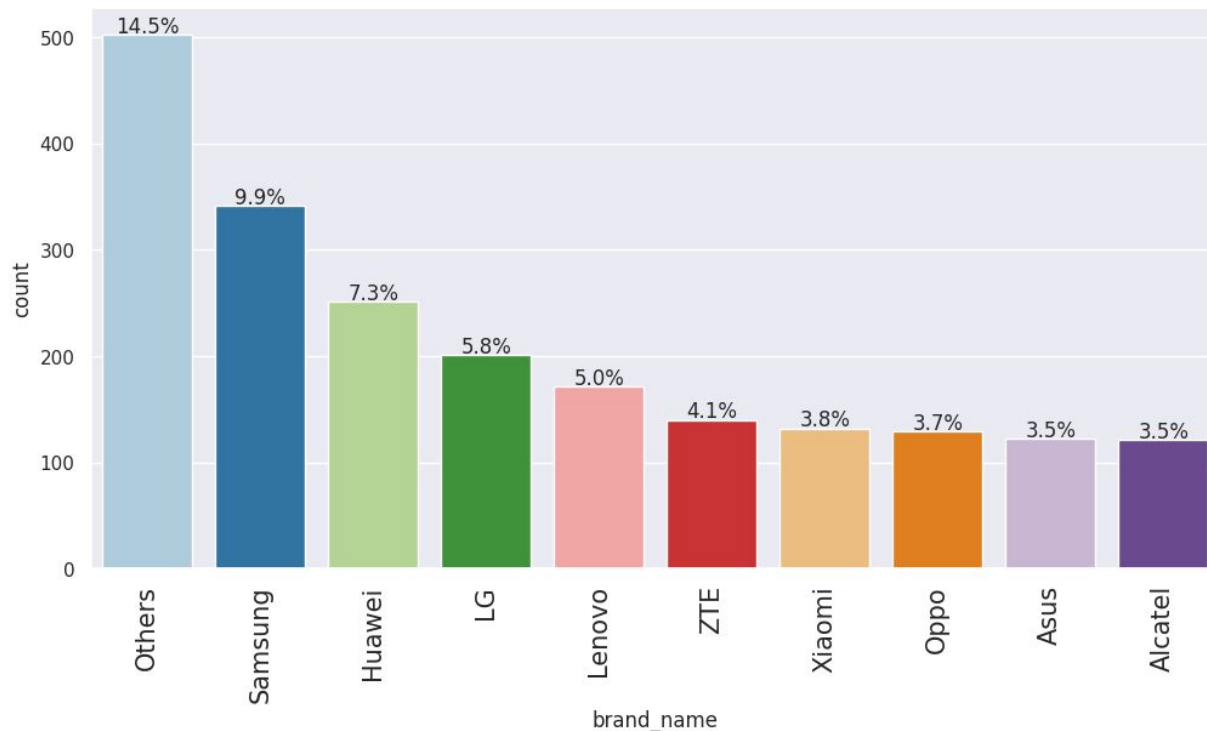


Observation:

No outliers observed;
Slightly left skewed
distribution with median at
690 days, and a mean of 674
days.

Name of manufacturing brand

```
labeled_barplot(df, "brand_name", perc=True, n=10)
```

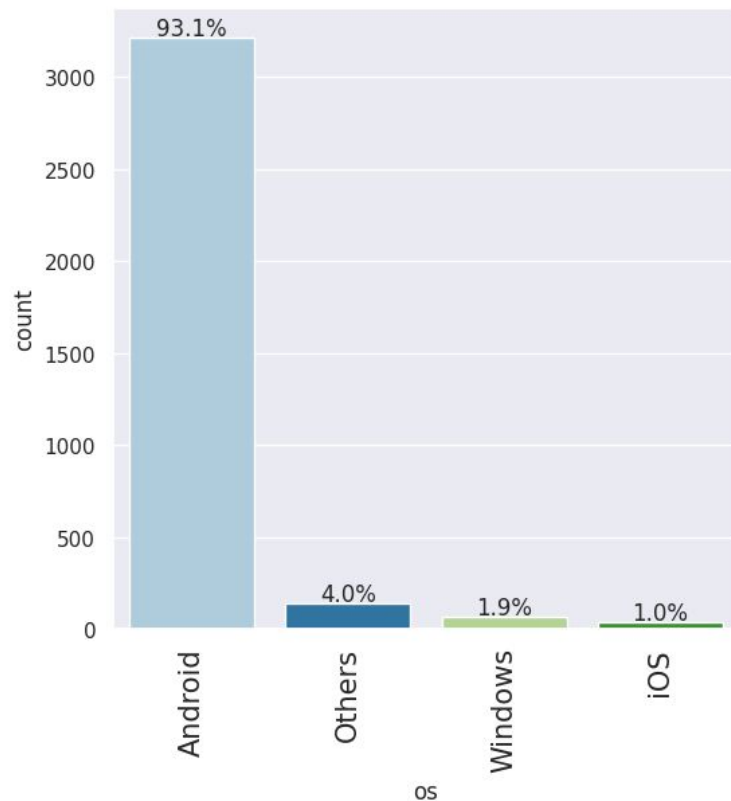


Observation:

The top 25% of most popular brand names are “Others” at 14.5%, and Samsung at 9.9%

OS on which the device runs

```
labeled_barplot(df, 'os', Perc=True)
```

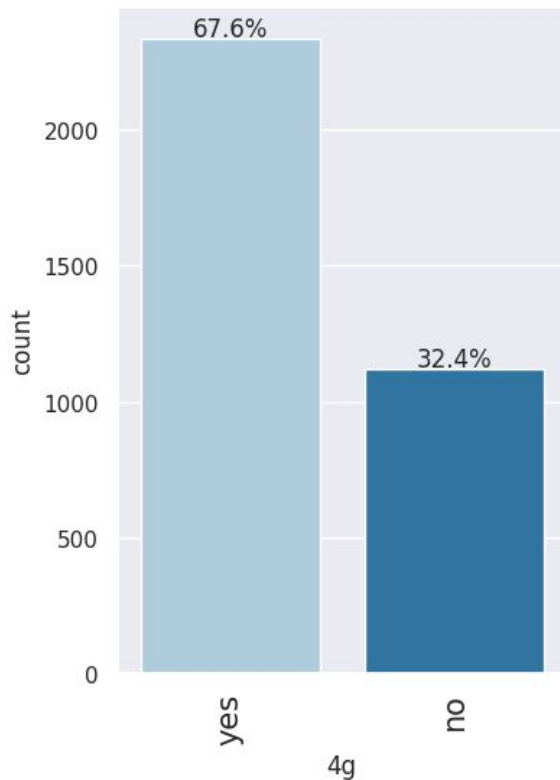


Observation:

The most popular operating system by far is Android with more than 93.1% of the counts.

Whether 4G is available or not

```
labeled_barplot(df, '4g', perc=True)
```

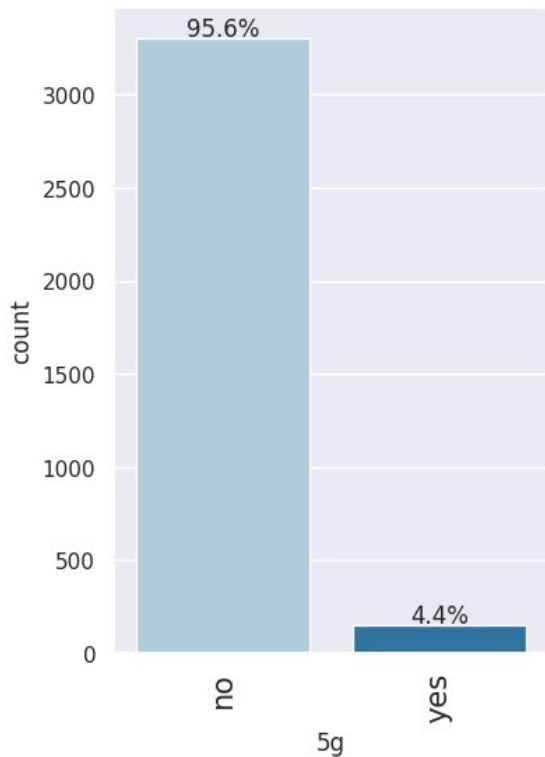


Observation:

67.6% of the used / refurbished devices have 4G available.

Whether 5G is available or not

```
labeled_barplot(df, '5g', perc=True)
```

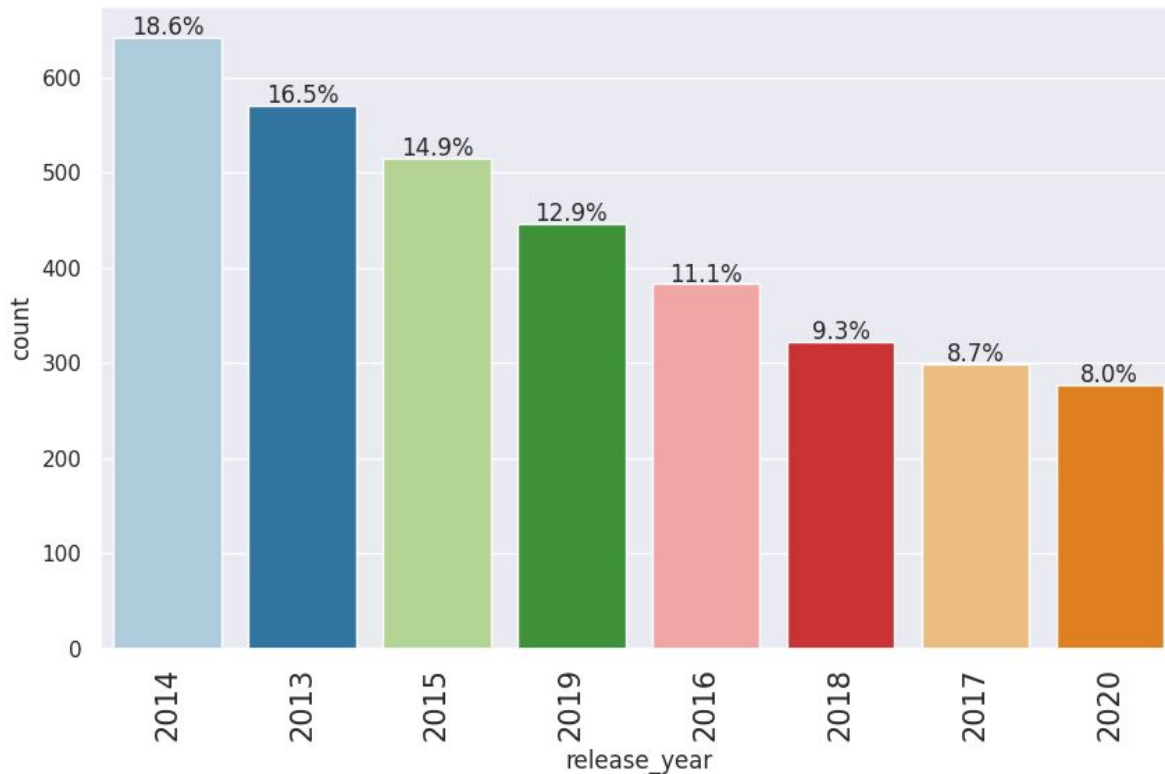


Observation:

95.6% of the devices do not have 5G available. Only 4.4% does.

Year when the device model was released

```
labeled_barplot(df, 'release_year', perc=True)
```



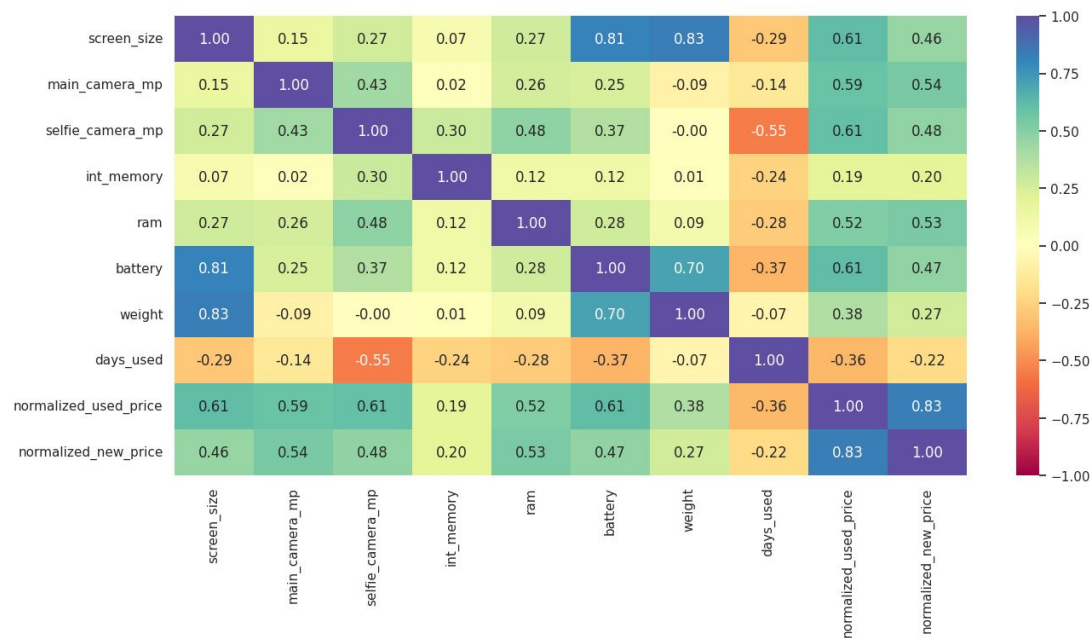
Observation:

The barplot shows an even gradual negative distribution over the years when a device model was released. Starting with 2014 at 18.6%, down 2020 at 8%.

EDA - Bivariate Analysis

Correlation Check

```
cols_list = df.select_dtypes(include=np.number).columns.tolist()
cols_list.remove("release_year") plt.figure(figsize=(15, 7))
sns.heatmap(df[cols_list].corr(), annot=True, vmin=-1, vmax=1, fmt=".2f", cmap="Spectral")
plt.show()
```



Observation:

Highly correlated are the normalized prices for new and used devices, which is to be expected since the used price is largely based on competitive pricing of the new models. High correlation among battery, weight, and screen size as well.

EDA - Bivariate Analysis II

Code for boxplot analysis of amount of RAM across brand names

```
plt.figure(figsize=(15, 5))  
sns.boxplot(data=df, x="brand_name", y="ram") plt.xticks(rotation=90)  
plt.show()
```

Code for boxplot analysis of large battery and weight across brands

```
df_large_battery = df[df.battery > 4500]  
df_large_battery.shape  
(341, 15)  
  
plt.figure(figsize=(15, 5))  
sns.boxplot(df_large_battery, x='brand_name', y='weight')  
plt.xticks(rotation=90)  
plt.show()
```

EDA - Bivariate Analysis III

Code for barplot analysis of devices with large screens across brands.

```
df_large_screen = df[df.screen_size > 6 * 2.54]
df_large_screen.shape (1099, 15)
labeled_barplot(df_large_screen, 'brand_name', perc=True)
```

Code for barplot analysis of devices with good front cameras:

```
df_selfie_camera = df[df.selfie_camera_mp > 8]
df_selfie_camera.shape (655, 15)
labeled_barplot(df_selfie_camera, 'brand_name', perc=True)
```

Code for barplot analysis of devices with high resolution rear cameras:

```
df_main_camera = df[df.main_camera_mp > 16]
df_main_camera.shape (94, 15)
labeled_barplot(df_main_camera, 'brand_name', perc=True)
```

EDA - Bivariate Analysis IV

Code for line plot analysis of price of used devices across the years.

```
plt.figure(figsize=(12, 5))  
sns.lineplot(data=df, x='release_year', y= 'normalized_used_price')  
plt.show()
```

Code for analysis on prices for used phones and tablets offering 4G and 5G networks.

```
plt.figure(figsize=(10, 4))  
plt.subplot(121)  
sns.boxplot(data=df, x="4g", y="normalized_used_price")  
plt.subplot(122)  
sns.boxplot(data=df, x="5g", y="normalized_used_price")  
plt.show()
```

Data preprocessing - Missing Value treatment code

- We will impute the missing values in the data by the column medians grouped by release_year and brand_name.

```
cols_impute = ["main_camera_mp", "selfie_camera_mp", "int_memory", "ram", "battery", "weight",]
```

```
for col in cols_impute:
```

```
    df1["main_camera_mp"] = df1["main_camera_mp"].fillna(
```

```
        value=df1.groupby(["brand_name", "release_year"])["main_camera_mp"].transform("median")
```

```
    ) ## Complete the code to impute missing values in cols_impute with median by grouping the data on  
    release year and brand name
```

```
# checking for missing values
```

```
df1["main_camera_mp"].isnull().sum() ## Complete the code to check missing values after imputing the  
above columns
```

179

Data preprocessing - Missing Value treatment code II

```
for col in cols_impute:
df1["selfie_camera_mp"] = df1["selfie_camera_mp"].fillna(
value=df1.groupby(["brand_name", "release_year"])["selfie_camera_mp"].transform("median")
) ## Complete the code to impute missing values in cols_impute with median by grouping the data on
release year and brand name

# checking for missing values
df1["selfie_camera_mp"].isnull().sum() ## Complete the code to check missing values after imputing the
above columns
```

2

```
for col in cols_impute:
df1['int_memory']=df1['int_memory'].fillna(
value=df1.groupby(['brand_name', 'release_year'])['int_memory'].transform('median')
)
df1['int_memory'].isnull().sum()
```

0

Data preprocessing - Missing Value treatment code III

```
for col in cols_impute:
```

```
    df1['ram']=df1['ram'].fillna(
```

```
        value=df1.groupby(['brand_name','release_year'])['ram'].transform('median')
```

```
    )
```

```
df1['ram'].isnull().sum()
```

output

0

```
for col in cols_impute:
```

```
    df1['battery']=df1['battery'].fillna(
```

```
        value=df1.groupby(['brand_name','release_year'])['battery'].transform('median')
```

```
    )
```

```
df1['battery'].isnull().sum()
```

output

6

Data preprocessing - Missing Value treatment code IV

```
for col in cols_impute:
    df1['weight']=df1['weight'].fillna(
        value=df1.groupby(['brand_name','release_year'])['weight'].transform('median')
    )
```

```
df1['weight'].isnull().sum()
```

output

7

- We will impute the remaining missing values in the data by the column medians grouped by brand_name.

```
cols_impute = [
    "main_camera_mp",
    "selfie_camera_mp",
    "battery",
    "weight",
]
```

Data preprocessing - Missing Value treatment code V

```
for col in cols_impute:

    df1["main_camera_mp"] = df1["main_camera_mp"].fillna(

        value=df1.groupby(['brand_name'])["main_camera_mp"].transform("median")

    ) ## Complete the code to impute the missing values in cols_impute with median by grouping the data
on brand name

# checking for missing values

df1["main_camera_mp"].isnull().sum() ## Complete the code to check missing values after imputing the
above columns

output
10
```

Data preprocessing - Missing Value treatment code VI

```
for col in cols_impute:
```

```
    df1["selfie_camera_mp"]=df1["selfie_camera_mp"].fillna(
```

```
        value=df1.groupby(["brand_name"])["selfie_camera_mp"].transform("median")
```

```
    )
```

```
df1["selfie_camera_mp"].isnull().sum()
```

output

0

```
for col in cols_impute:
```

```
    df1["battery"]=df1["battery"].fillna(
```

```
        value=df1.groupby(["brand_name"])["battery"].transform("median")
```

```
    )
```

```
df1["battery"].isnull().sum()
```

output

0

Data preprocessing - Missing Value treatment code VII

```
for col in cols_impute:
```

```
    df1["weight"] = df1["weight"].fillna(
```

```
        value=df1.groupby(["brand_name"])["weight"].transform("median")
```

```
    )
```

```
df1["weight"].isnull().sum()
```

output

0

- We will fill the remaining missing values in the main_camera_mp column by the column median.

```
df1["main_camera_mp"] = df1["main_camera_mp"].fillna(df1["main_camera_mp"].median()) ## Complete the  
code to impute the data with median
```

```
# checking for missing values
```

```
df1["main_camera_mp"].isnull().sum() ## Complete the code to check missing values after imputing the  
above columns
```

output

0

Data preprocessing - Feature engineering

```
df1["years_since_release"] = 2021 - df1["release_year"]
```

```
df1.drop("release_year", axis=1, inplace=True)
```

```
df1["years_since_release"].describe()
```

output

```
count    3454.000000
mean         5.034742
std         2.298455
min         1.000000
25%         3.000000
50%         5.500000
75%         7.000000
max         8.000000
```

```
Name: years_since_release, dtype: float64
```

Data preprocessing - Outlier check

```
# outlier detection using boxplot
num_cols = df1.select_dtypes(include=np.number).columns.tolist()

plt.figure(figsize=(15, 15))

for i, variable in enumerate(num_cols):
    plt.subplot(4, 3, i + 1)
    sns.boxplot(data=df1, x=variable)
plt.tight_layout(pad=2)

plt.show()
```


Model Assumptions - Multicollinearity removal cont'd

- Drop every column one by one that has a VIF score greater than 5.

```
col_to_drop = 'os_iOS' ## Complete the code to specify the column to drop
x_train2 = x_train.loc[:, ~x_train.columns.str.startswith(col_to_drop)] ## Complete the code to specify
the train data from which to drop the column specified
x_test2 = x_test.loc[:, ~x_test.columns.str.startswith(col_to_drop)] ## Complete the code to specify the
test data from which to drop the column specified

# Check VIF now
vif = checking_vif(x_train2)
print("VIF after dropping ", col_to_drop)
Vif

col_list = ['screen_size', 'weight'] ## Complete the code to specify the columns with high VIF
```

Model Assumptions - Multicollinearity removal cont'd

- Look at the adjusted R-squared and RMSE of all these models.

```
res = treating_multicollinearity(x_train2, y_train, col_list) ## Complete the code to check the effect on  
model performance after dropping specified columns from train data  
res
```

col	Adj. R-squared after_dropping col	RMSE after dropping col	
0	screen_size	0.838363	0.234716
1	weight	0.838139	0.234879

Model Assumptions - Multicollinearity removal cont'd

```
col_to_drop= 'weight' ## Complete the code to specify the column to drop
x_train3 = x_train2.loc[:, ~x_train2.columns.str.startswith(col_to_drop)] ## Complete the code to specify
the train data from which to drop the column specified
x_test3 = x_test2.loc[:, ~x_test2.columns.str.startswith(col_to_drop)] ## Complete the code to specify
the test data from which to drop the column specified

# Check VIF now
vif = checking_vif(x_train2)
print("VIF after dropping ", col_to_drop)
vif
```

Model Assumptions - Multicollinearity removal cont'd

```
col_list = ['screen_size'] ## Complete the code to specify the columns with high VIF
```

```
res = treating_multicollinearity(x_train3, y_train, col_list) ## Complete the code to check the effect on  
model performance after dropping specified columns from train data
```

Res

	Col	Adj. R-squared after_dropping col	RMSE after dropping col
0	screen_size	0.815336	0.250879

```
col_to_drop = 'screen_size' ## Complete the code to specify the column to drop
```

```
x_train4 = x_train3.loc[:, ~x_train3.columns.str.startswith(col_to_drop)] ## Complete the code to specify  
the train data from which to drop the column specified
```

```
x_test4= x_test3.loc[:, ~x_test3.columns.str.startswith(col_to_drop)] ## Complete the code to specify the  
test data from which to drop the column specified
```

```
# Check VIF now
```

```
vif = checking_vif(x_train2)
```

```
print("VIF after dropping ", col_to_drop)
```

```
vif
```

Model Assumptions - checking on p-values

```
#We have dealt with multicollinearity in the data  
#Let's rebuild the model using the updated set of predictors variables  
olsmodel1 = sm.OLS(y_train, x_train4).fit() ## Complete the code to fit OLS model  
print(olsmodel1.summary())
```

Observations on the OLS model1:

We can see that adj. R-squared has dropped from 0.842 to 0.815, which shows that the dropped columns did not have much effect on the model. As there is no multicollinearity, we can look at the p-values of predictor variables to check their significance.

Dropping high p-value variables (if needed)

- We will drop the predictor variables having a p-value greater than 0.05 as they do not significantly impact the target variable.
- But sometimes p-values change after dropping a variable. So, we'll not drop all variables at once.

Model Assumptions - checking on p-values cont'd

```
# initial list of columns
predictors = x_train4.copy() ## Complete the code to check for p-values on the right dataset
cols = predictors.columns.tolist()

# setting an initial max p-value
max_p_value = 1

while len(cols) > 0:
    # defining the train set
    x_train_aux = predictors[cols]

    # fitting the model
    model = sm.OLS(y_train, x_train_aux).fit()

    # getting the p-values and the maximum p-value
    p_values = model.pvalues
    max_p_value = max(p_values)
```

Model Assumptions - checking on p-values cont'd

```
# name of the variable with maximum p-value
```

```
feature_with_p_max = p_values.idxmax()
```

```
if max_p_value > 0.05:
```

```
    cols.remove(feature_with_p_max)
```

```
    print(f"DROPPED: feature {feature_with_p_max} with p-value {max_p_value}")
```

```
else:
```

```
    break
```

```
selected_features = cols
```

```
print(selected_features)
```

```
x_train5 = x_train4[selected_features] ## Complete the code to specify the train data from which to  
select the specified columns
```

```
x_test5 = x_test4[selected_features] ## Complete the code to specify the test data from which to select  
the specified columns
```

Model Assumptions - checking on p-values cont'd

```
olsmod2 = sm.OLS(y_train, x_train5).fit() ## Complete the code fit OLS() on updated dataset (no
multicollinearity and no insignificant predictors)
print(olsmod2.summary())

# checking model performance on train set (seen 70% data)
print("Training Performance\n")
olsmod2_train_perf = model_performance_regression(olsmod2, x_train5, y_train) ## Complete the code to
check performance on train data
olsmod2_train_perf
```

Training Performance

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	0.250287	0.194316	0.81613	0.815135	4.677589

Model Assumptions - checking on p-values and build 2nd Version of model

(note: I made a 2nd model that turns out to be the same as final model PZ)

```
# checking model performance on test set (seen 30% data)
```

```
print("Test Performance\n")
```

```
olsmod2_test_perf = model_performance_regression(olsmod2, x_test5, y_test) ## Complete the code to check  
performance on test data
```

```
olsmod2_test_perf
```

Test Performance

	RMSE	MAE	R-squared	Adj. R-squared	MAPE
0	0.252521	0.195275	0.823203	0.820956	4.778809

Model assumptions continued - test for linearity and independence

```
# let us create a dataframe with actual, fitted and residual values
```

```
df_pred = pd.DataFrame()
```

```
df_pred["Actual Values"] = y_train # actual values
```

```
df_pred["Fitted Values"] = olsmod2.fittedvalues # predicted values
```

```
df_pred["Residuals"] = olsmod2.resid # residuals
```

```
df_pred.head()
```

	Actual Values	Fitted Values	Residuals
3026	4.087488	3.912528	0.174960
1525	4.448399	4.596369	-0.147969
1128	4.315353	4.290987	0.024366
3003	4.282068	4.307938	-0.025870
2907	4.456438	4.436581	0.019857

Slide Header

- Please mention any other pointers (if needed)



Happy Learning !

